

ASM and Datapath

Suppose we are asked to design a sequential circuit that will multiply two four-bit numbers and give an eight-bit result. Here is a familiar approach that will work:

$$\begin{array}{r} 1011 \\ \times 0101 \\ \hline 1011 \\ 0000 \\ 1011 \\ 0000 \\ \hline 0110111 \end{array}$$

It IS a finite state machine, so we **could** proceed in the usual way, but this is a BAD plan. Consider:

Q0 depends on **(X0, Y0)**

Q1 depends on **(X0, Y0, X1, Y1)**

Q2 depends on **(X0, Y0, X1, Y1, X2, Y2)**

Q3-Q7 depend on **(X0, Y0, X1, Y1, X2, Y2, X3, Y3)**

Remember that each state, in general, has two arcs out of it for each input bit. So by the time we get to Q3, we are defining **sixteen** paths out of each state.

And how many states? In principle, we need one for each possible permutation of each column: **4x8x8x8**

OK, this is a bad plan.

Datapath

A much more sensible approach is to separate out the **datapath** from the **control logic**.

By **datapath** we mean the parts of the circuit that manage the **content** of what we are processing. Very often, we can separate “what we are processing” from “how we are processing it”.

Multiplication (again)

$$\begin{array}{r}
 1011 \\
 \times 0101 \\
 \hline
 1011 \\
 0000 \\
 1011 \\
 0000 \\
 \hline
 0110111
 \end{array}$$

The trick is to realize that each multiplication is actually a special case of this:

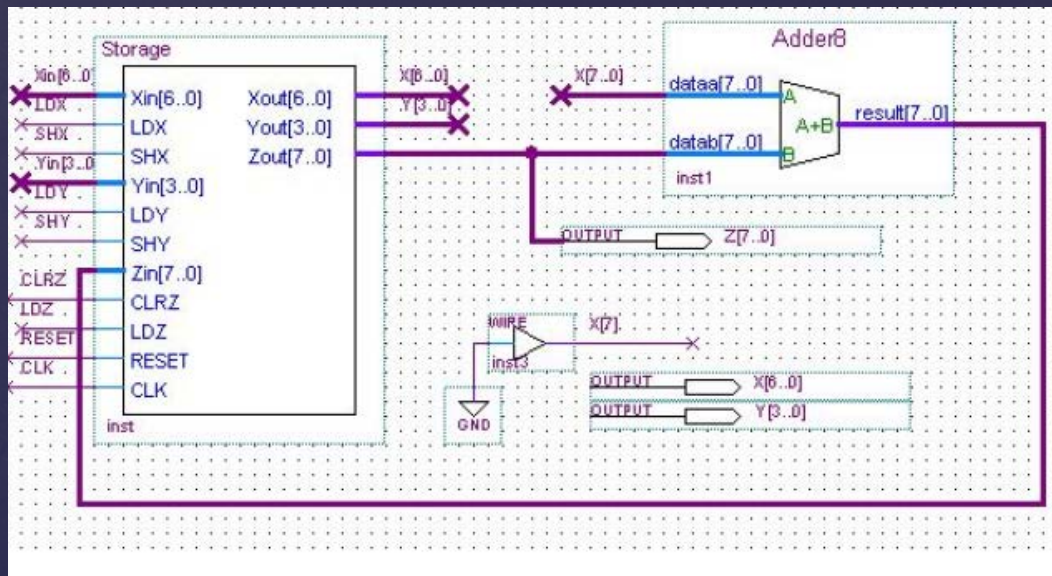
$$\begin{array}{r}
 X_3 X_2 X_1 X_0 \\
 \times Y_3 Y_2 Y_1 Y_0 \\
 \hline
 X_3 X_2 X_1 X_0 \bullet Y_0 \\
 X_3 X_2 X_1 X_0 \bullet Y_1 \\
 X_3 X_2 X_1 X_0 \bullet Y_2 \\
 + X_3 X_2 X_1 X_0 \bullet Y_3 \\
 \hline
 \end{array}$$

Assuming the right combination of adders, counters and shift registers...

The only “control” is “Do I add on this clock cycle or not?”

Multiplication (cont)

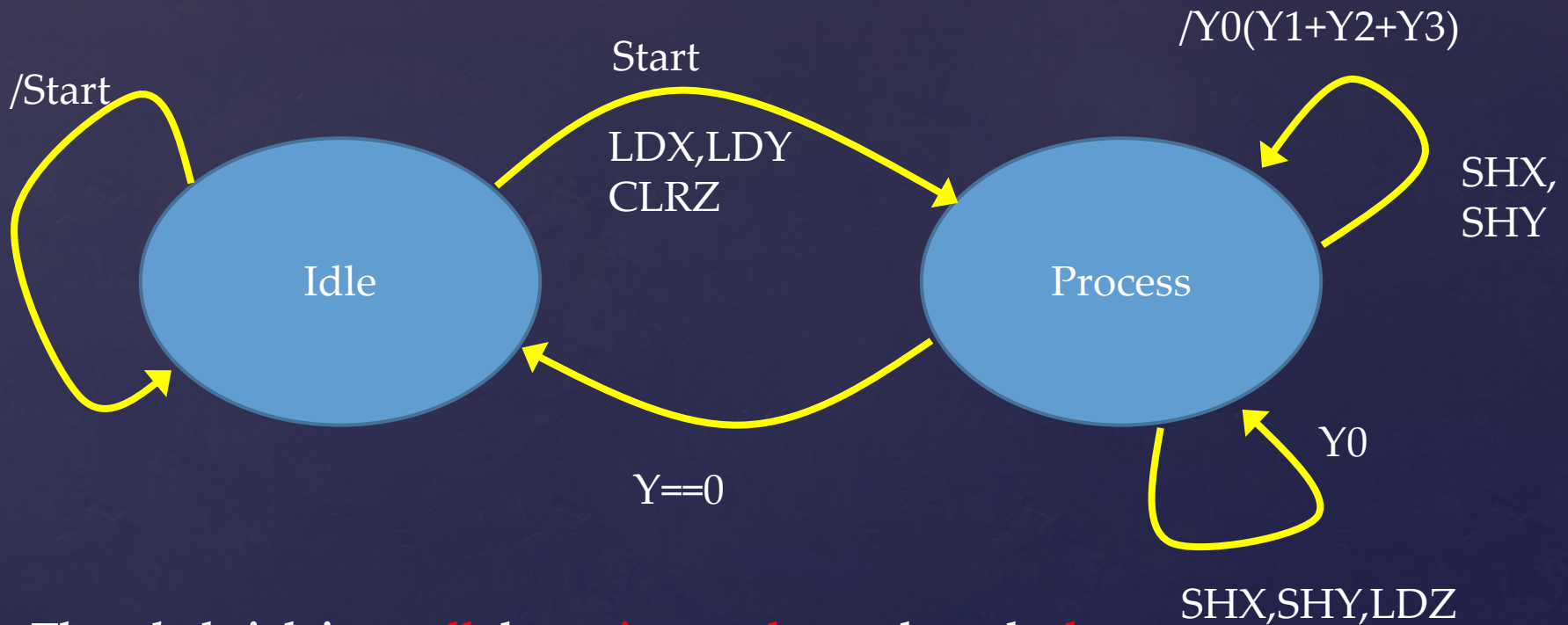
The trick is: the **data** isn't really an **input** to the state machine. Select **conditions** about the data at **specific times** can be. We can define the **datapath** like this:



The storage is:
X: 7-bit Left Shifter
Y: 4-bit Right Shifter
Z: 8-bit Register

Multiplication (cont)

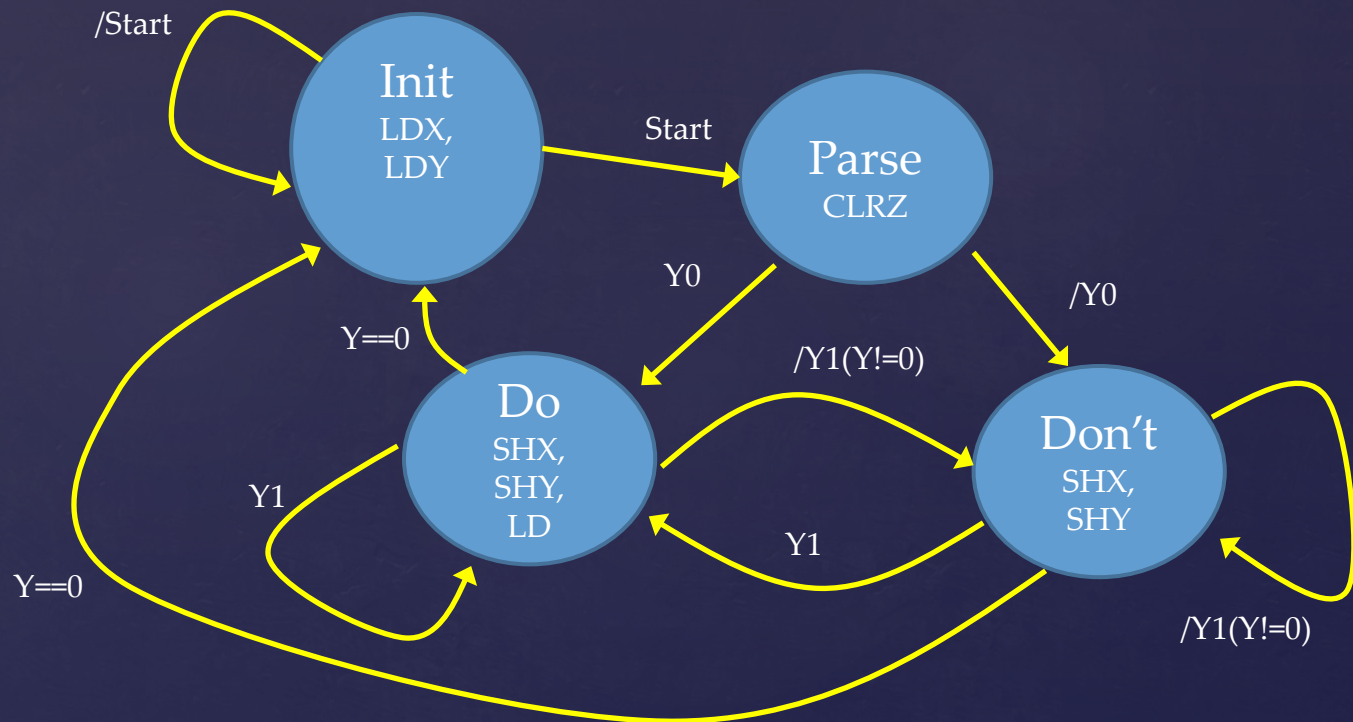
With this separation, the “FSM” part is simple:



The whole job is to **tell** the **registers what** to do and **when**.

Multiplication (cont)

As usual, a Moore-model version needs more states and more time:



A note on timing

It is easy to become confused about the timing inside a synchronous ASM. We have one synchronous machine (the **control logic**) generating signals for another synchronous machine (the **datapath**), usually with feedback from the datapath affecting the control logic.

Again: who does what, when?

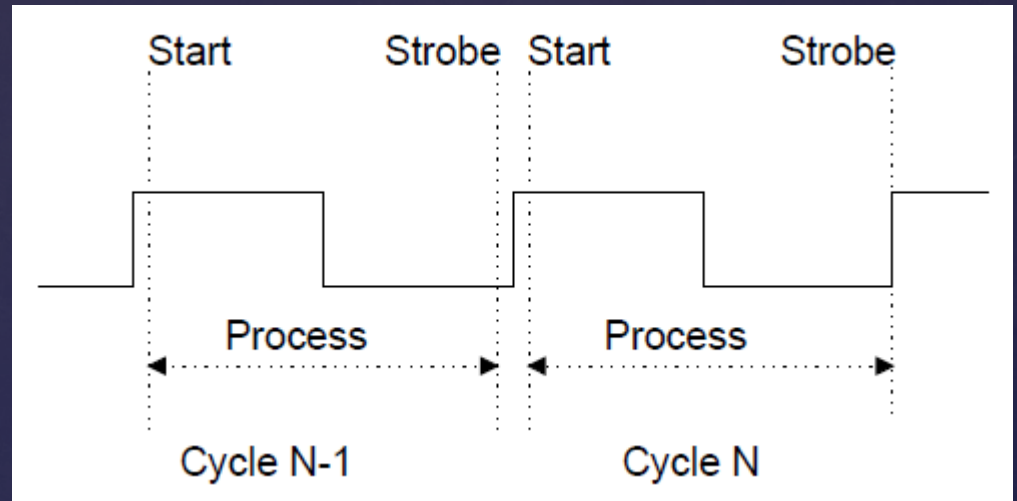
Timing (cont)

I like to mentally divide each clock cycle into three parts:

Start Register data are available **just after** the rising edge of the clock

Process Most of the clock cycle is for computation and/or data movement

Strobe Registers **strobe** (take a snapshot of) the input **just before** the next rising edge



Timing (cont)

That way, signals to registers all live in “this” clock cycle:

OE OE (Output Enable) means “let your data out at the instant of **Start**”

ALU Computational instructions (ADD/SUB/INC/ETC) take effect during the **Process** time

LD Signals that **change** register data take effect at **Strobe**
SH
CNT

Let's do another one

Asynchronous serial communication (UART) works like this:

1. Idle high
2. One START bit
3. Eight DATA bits (from a register)
4. One STOP bit

What is the datapath? What does the control logic do?

Intro to Microcoding

Here is the state-and-output table for our UART:

State	Input	Next State	Output
IDLE	/START	IDLE	CLR,SEL[]=1
IDLE	START	SBIT	CLR,SEL[]=1
SBIT	ALL	TX	SH,LD,CLR
TX	/EQ7	TX	SH,CNT,SEL[]=2
TX	EQ7	STOP	SH,CNT,SEL[]=2
STOP	ALL	IDLE	SEL[]=1

Intro to Microcoding

This state table has a nice (but rare!) feature. Note that there are never more than two paths out of a state. (That rarely occurs naturally, but you can force it). Let's make some definitions:

State	s_1s_0
IDLE	00
SBIT	01
TX	10
STOP	11

Conditions	Logic
C1	/START
C2	/EQ7
C3	VCC

Output bit	Meaning
O_5	SH
O_4	LD
O_3	CLR
O_2	CNT
O_1	SEL[1]
O_0	SEL[0]

Intro to Microcoding

With these definitions, here is our table again:

State s_1s_0	Next State S_1S_0	Condition	Output $O_5O_4O_3O_2O_1O_0$
00	00	None	001001
00	01	C0	001001
01	10	C2	111000
10	10	None	100110
10	11	C1	100110
11	00	C2	000001

Intro to Microcoding

Here's one more way to look at it:

State s_1s_0	Next State Logic	Output $O_5O_4O_3O_2O_1O_0$
00	if (C0) 00 else 01	001001
01	if (C2) 10 else 10	111000
10	if (C1) 10 else 11	100110
11	if (C2) 00 else 00	000001

Intro to Microcoding

Or, if we encode the **number** of the condition rather than its name:

State s_1s_0	Condition C_1C_0	Target A_1A_0	Output $O_5O_4O_3O_2O_1O_0$
00	00	00	001001
01	10	10	111000
10	01	10	100110
11	10	00	000001

Where did the “else” part go? We don’t need it. It’s just “next”

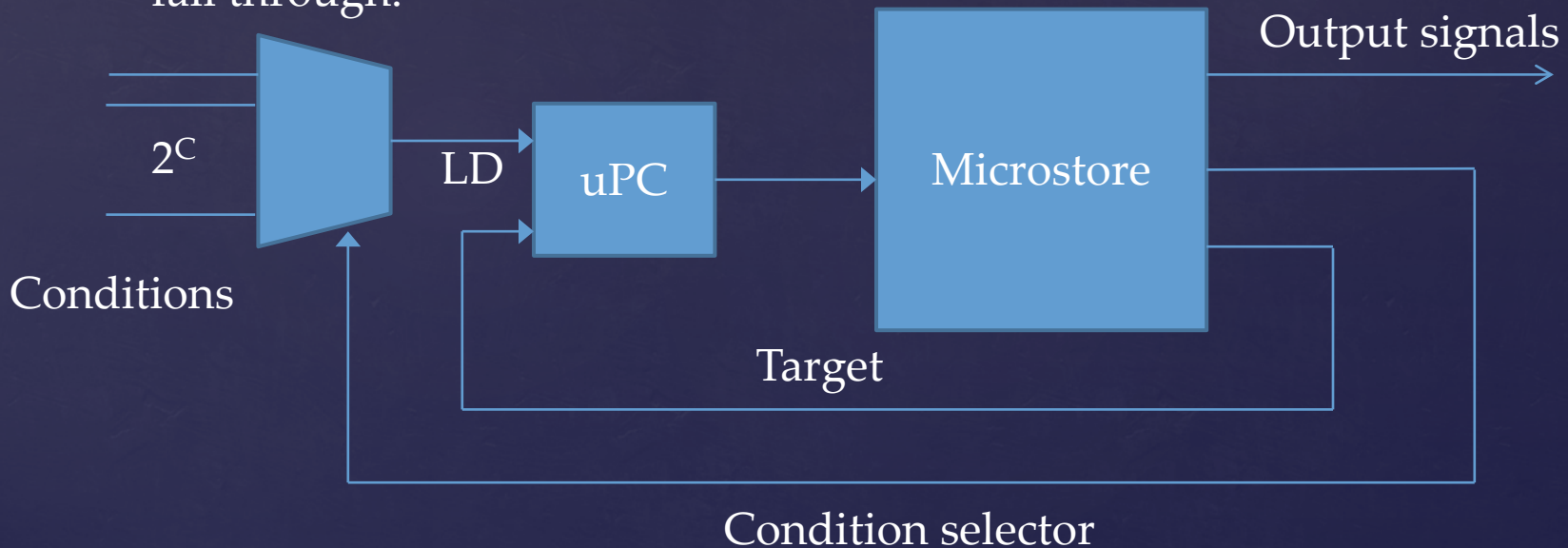
Microinstructions

Here's the punchline. This isn't a "state machine" anymore. It's a **program**

We call this a **microprogram**, made of **microinstructions**. Each microinstruction encodes the **output** (control logic), a **branch target** and **selector bits** to choose a condition. If the condition is false, we just fall through to the next microinstruction.

Microcode Sequencer

A typical microcode system includes a **microstore (tiny ROM)**, a **microprogram counter** and a **multiplexer**. The microstore holds **microinstructions**. Each microinstruction specifies the output, the condition and the target. The selected condition decides if we branch or fall through.



Let's do another one

Let's try to adapt our sequential multiplier (Moore version) to this approach?

Is the state machine suitable? What has to change?